3. Add a unique identifier to your .csproj file. Visual Studio does this automatically when you choose Manage User Secrets, but if you're using the command line, you'll need to add it yourself. Typically, you'd use a unique ID, such as a globally unique identifier (GUID):

```
<PropertyGroup>
  <UserSecretsId>96eb2a39-1ef9-4d8e-8b20-8e8bd14038aa</UserSecretsId>
</PropertyGroup>
```

   You can also generate the `UserSecretsId` property with a random value using the .NET command-line interface (CLI) by running the following command from your project folder:

```
dotnet user-secrets init
```

4. Add User Secrets by using the command line

```
dotnet user-secrets set "MapSettings:GoogleMapsApiKey" F5RJT9GFHKR7
```

   or edit the secret.json file directly by using your favorite editor. The exact location of this file depends on your operating system and may vary. Check the documentation for details at http://mng.bz/ryAg.

NOTE The Secret Manager tool is included in the .NET CLI, but you can also use the CLI to install additional .NET tools. You can find more about .NET tools in general in Microsoft's "How to manage .NET tools" documentation: http://mng.bz/VdmX.

Phew! That's a lot of setup, and if you're adding providers to `ConfigurationManager` manually, you're not done yet! You need to update your app to load the User Secrets at runtime by using the `AddUserSecrets` extension method:

```
if (builder.Environment.IsDevelopment())
{
    builder.Configuration.AddUserSecrets<Program>();
}
```